Introduction to Python

Week 5

Program for This Week: Data analysis with Pandas

1. Dataframe
2. Series
3. Selection
4. Missing data
5. Operations on data
6. Merging (concatenate and join)
7. Grouping

Merging and grouping are part of the weekly exercises but are not exam material.

Pandas DataFrame

- Pandas is one of the standard libraries in data science
- The most important datatype used in Pandas is the *DataFrame*
  - A DataFrame contains tabular data just like a spreadsheet in excel
  - A Dataframe with only one column is called a Series
- Pandas is built on top of NumPy, so there is overhead because of the conversion to and from NumPy structures
- NumPy lacks the flexibility of Python, and therefore is fast when crunching numbers
- Pandas works fast with uniform numerical data, but can be slow when working with string data
- Pandas is very flexible, this comes with a performance price
- Like NumPy, Pandas offers *vectorization:* you can apply a function to all elements of a column/row/frame. Vectorization will probably be faster than loops or the map function in Python, but that is not a given
- Vectorization can make your code more readable
- An interesting library for data science is Polar, written in Rust, avoiding Pandas/Numpy conversion overhead. https://www.datacamp.com/blog/an-introduction-to-polars-python-s-tool-for-large-scale-data-analysis.
- Interesting is also Microsoft's project 'Python in excel' where your interface is not a Pandas DataFrame but an excel spreadsheet. https://support.microsoft.com/en-us/office/introduction-to-python-in-excel-55643c2e-ff56-4168-b1ce-9428c8308545

DataFrame, rownames and colnames

	Name	Length	Weight
First	A	153	55
Second	B	160	70
Third	C	150	60

- Name, Length and Weight are the names of the *columns* and First, Second, and Third are the names of the *rows* (also called *indices*)
- In these slides we will call these *colnames* and *rownames*, respectively, and you can get these the following way:
  - `print(df.index)  # Onscreen: Index(['First', 'Second', 'Third'], dtype='object')`
  - `print(df.columns)  # Onscreen: Index(['Name', 'Length', 'Weight'], dtype='object')`
- `df.index` and `df.columns` are special attributes, over which you can iterate, and apply slicing and indexing to
- These objects also have a map method, which you can use to apply a function to their elements
- Elements of these objects cannot be changed, but they can be overwritten in full:
  - `df.index[1] = 'fourth'  # Onscreen: Error`
  - `df.index = ['First', 'Fourth', 'Third']  # Works`

Creating a DataFrame using a dictionary

- Say you want to create the following dataframe:

	Name	Length	Weight
First	A	153	55
Second	B	160	70
Third	C	150	60

- One of the easiest ways to create this dataframe is by using a dictionary:

```
import pandas as pd
d1 = {'Name':['A', 'B', 'C'],'Length':[153, 160, 150],'Weight':[55, 70, 60]}
df1 = pd.DataFrame (d1, index=['First', 'Second', 'Third'])
print(df1)
```

Creating a DataFrame using a dictionary

- Let's do some closer inspection of the code, line by line:
- `import pandas as pd`
  - You need to import the Pandas package just like the math package as it is not standard imported in Python
  - We rename the package to `pd`, this is simply a convention
- `d1 = {'Name':['A','B','C'],'Length':[153,160,150],'Weight':[55,70,60]}`
  - We create a dictionary, where each key:value pair represents a column: the keys are the colnames and the values of the columns are in the sublists
- `df1 = pd.DataFrame (d1, index=['First','Second','Third'])`
  - We use the DataFrame class from the Pandas package to create a dataframe object and give it the name `df1`
  - The index parameter defines the rownames
- `print(df1)`
  - Print the created dataframe object to see whether it turned out as we planned

Creating a DataFrame using a list of lists

- Alternatively, you can create a dataframe using a list of lists, where every sublist represents a row
- ```
  import pandas as pd
  l1 = [['A', 153, 55], ['B', 160, 70], ['C', 150, 60]]
  df1 = pd.DataFrame (l1, index=['First', 'Second',
  'Third'], columns=['Name', 'Length', 'Weight'])
  print(df1)
  ```
- Since there are no keys, you must pass the names of the columns another way to the DataFrame class
- Instead of a list of lists, you can use a tuple of lists, or a list of tuples, or tuples of tuples
  ```

Creating a DataFrame using the transpose operator

- ```
  import pandas as pd
  d1 = {'First':['A', 153, 55],'Second':['B', 160, 70],'Third':['C', 150, 60]}
  df1 = pd.DataFrame (d1, index=['Name', 'Length', 'Weight'] )
  ```
- `print(df1)`

	First	Second	Third
Name	A	B	C
Length	153	160	150
Weight	55C	70	60

- `df1 = df1.T`
- `print(df1)`

	Name	Length	Weight
First	A	153	55
Second	B	160	70
Third	C	150	60
  ```

Creating a DataFrame using default col and row names

- ```
  import pandas as pd
  l1 = [['A', 153, 55], ['B', 160, 70], ['C', 150, 60]]
  df1 = pd.DataFrame (l1)
  ```
- The result of this code will be:

	0	1	2
0	A	153	55
1	B	160	70
2	C	150	60

- You might think they look a lot like indices from a list, and it does, but looks can be deceiving since they are colnames and rownames, and are more like keys in dictionaries
  ```

Creating a series using a dictionary or a list

- Say you want to create the following series

First	153
Second	160
Third	150
Name: Length	

- ```
  import pandas as pd
  d1 = {'First': 153, 'Second': 160, 'Third': 150}
  s1 = pd.Series(d1, name = 'Length')
  print(s1)
  ```
- ```
  import pandas as pd
  l1 = [153, 160, 150]
  s1 = pd.Series(l1, name = 'Length', index=['First', 'Second', 'Third'])
  print(s1)
  ```

Series using defaults

- ```
  import pandas as pd
  l1 = [153, 160, 150]
  s1 = pd.Series(l1)
  print(s1)
  ```
- The result of this code will be:

0	153
1	160
2	150
  ```

Dataframe and Series indices are like keys in dictionaries

- With a list you can do the following:
- ```
  l1 = [153, 160, 150]
  del l1[1]
  del l1[1]
  print (l1)            # Onscreen: [153]
  ```
- You cannot do that with a dictionary:
- ```
  d1 = {0:153, 1:160, 2:150}
  del d1[1]
  del d1[1]             # Onscreen: KeyError: 1
  ```
- And, not with a series (or dataframe):
- ```
  s1 = pd.Series([153, 160, 150])
  s1 = s1.drop(1)
  s1 = s1.drop(1)       # Onscreen: KeyError: '[1] not found in axis'
  ```
- The reason is that de indices of a list give relative positions and are recalculated, while indices in a series (and in a dataframe), and keys in a dictionary are not recalculated
  ```

Viewing data

- Assume you created a dataframe like this:
  - ```
    import pandas as pd
    l1 = ['A', 'B', 'C', 'D'] * 250
    l2 = [160, 170, 180, 190, 200] * 200
    l3 = [55, 66] * 500
    d1 = {'Name':l1, 'Length': l2, 'Weight': l3}
    df1 = pd.DataFrame (d1)
    ```
- If you do: `print(df1)`, Python will only show you the first 5 and the last 5 rows
- To have a bit more control, you could use:
  - `print(df1.head(3))` to print the first 3 rows
  - `print(df1.tail(7))` to print the last 7 rows
  - You can use 'any' number as an argument in `head()` and `tail()`, but the default is 5
    ```

Viewing data

- If you are more interested in some quick statistics, you can use `describe()`
- We can apply this to our dataframe with 1000 rows: `print(df1.describe())`
- This results in:
  -
```
            Length       Weight
count 1000.000000 1000.000000
mean   180.000000   60.500000
std     14.149212    5.502752
min    160.000000   55.000000
25%    170.000000   55.000000
50%    180.000000   60.500000
75%    190.000000   66.000000
max    200.000000   66.000000
```

Sorting dataframes

1. Sorting a dataframe by rownames
2. Sorting a dataframe by colnames
3. Sorting a dataframe by the values in a row
4. Sorting a dataframe by the values in a column

Sorting a dataframe by rownames

- You have the following dateframe:

	Name	Length	Weight
First	A	153	55
Second	B	160	70
Third	C	150	60

- `df1 = df1.sort_index(axis=0, ascending=False))`

	Name	Length	Weight
Third	C	150	60
Second	B	160	70
First	A	153	55

- `axis = 0`, and `ascending = True` are the defaults

Sorting a dataframe by colnames

- You have the following dateframe:

	Name	Length	Weight
First	A	153	55
Second	B	160	70
Third	C	150	60

- `df1=df1.sort_index(axis=1, ascending=False)`

	Weight	Name	Length
First	55	A	153
Second	70	B	160
Third	60	C	150

- `ascending = True` is the default

Sorting a dataframe by the values in a column

- You have the following dateframe:

	Name	Length	Weight
First	A	153	55
Second	B	160	70
Third	C	150	60

- `df1=df1.sort_values(by='Length', axis=0, ascending=False)`

	Name	Length	Weight
Second	B	160	70
First	A	153	55
Third	C	150	60

  - This only works if all elements in the column have datatypes for which the > operator works
  - You can pass a list with several colnames to the by parameter, and several booleans to the ascending parameter
  - `axis = 0`, and `ascending = True` are the defaults

Sorting a dataframe by the values in a row

- You have the following dateframe:

	Name	Length	Weight
First	100	153	55
Second	200	160	70
Third	300	150	60

- `df1=df1.sort_values(by='First', ascending=False, axis=1)`

	Length	Name	Weight
First	153	100	55
Second	160	200	70
Third	150	300	60

  - This only works if all elements in a row have datatypes for which the > operator works
  - You can pass a list with several rownames to the by parameter, and several booleans to the ascending parameter
  - `axis = 0`, and `ascending = True` are the defaults

Selecting from a dataframe using columns

- You can use the column names to select the corresponding column, similar to using keys in a dictionary:
- `df1 = df1['Length']`
- `df1 = df1[['Length','Width']]`
- Notice that several columns are passed as a list!

Selecting from a dataframe, using loc

- `loc` works on the rownames and the colnames of the dataframe
- `df1.loc[row_selection, col_selection]`
  - Between the square brackets there are two parts:
    - Before the comma, you give your selection of rownames
    - After the comma, you give your selection of colnames
- `row_selection` can be filled in in different ways:
  - Slicing: `first_name : last_name`
    - You select from the rows from the top to the bottom
    - Your selection is a list of rownames: the first one `first_name`, the last one `last_name` (inclusive!) and all the rows in between
    - If you leave out `first_name` the default is the first row and if you leave out the `last_name` the default is the last row
    - So, if you leave out `first_name` and `last_name`, you select all the rows
  - A list with 1 or more rownames: e.g. `['First', Third']`
  - One rowname: e.g. `'First'`

Selecting from a dataframe: loc

- `col_selection` can be filled in in exactly the same ways:
  - Slicing: `first_name : last_name`
    - You make a selection from the columns from left to right
    - Your selection is a list of colnames: the first one `first_name`, the last one `last_name` (inclusive!) and all the columns in between
    - If you leave out `first_name` the default is the first column and if you leave out the `last_name` the default is the last column
    - So, if you leave out `first_name` and `last_name`, you select all the columns
  - A list with 1 or more colnames: e.g. `['Second', 'Third']`
  - One colname: e.g. `'Third'`
- The only difference between `row_selection` and `col_selection` is that you can leave out the `col_selection`
  - The default is all colnames

Selecting from a dataframe, using iloc

- `iloc` works almost the same as `loc`, but there are important differences:
  - `iloc` doesn't work with colnames and rownames, but with their relative position, like an index in a list
  - When slicing with `iloc` the end is **not** inclusive, like an index in a list
- Just like with lists you can use -1 etc. to count from the opposite corner
- You can use negative numbers when slicing, but also when you select rows/columns with a list of index numbers

Aside on slicing and indexing (Dijkstra)

- Slices could be `[inclusive:inclusive]`, `[inclusive:exclusive]`, `[exclusive:inclusive]` or `[exclusive:exclusive]`.
- `[in:ex]` and `[ex:in]`: length is difference (`len([5:18])=18-5`), and consecutive slices have overlapping point `([5:12] + [12:18] == 5:18])`
- `[ex:in]` and `[ex:ex]`: starting at zero means `[-1:…]` (ugly, and already used)
- `[in:in]` and `[ex:in]`: empty list is `[n:n-1]`. Very ugly.
- So `[in:ex]` it is.
- Given that, index of the last element of `[:n]` is `n` if we start indexing at 0, but it is `n` if we start indexing at 1.
- So we start indexing at 0.
- (Argument doesn't apply whenslicing by names instead of numbers. But still annoying…)

Selecting from a dataframe

- The result of a selection is quite intuitive
  - All the cells that are at the same time part of one of the selected columns and one of the selected rows are selected
- What may be less intuitive is the resulting datatype:

Row_selection	Col_selection	Result
A list/slice of rows	A list/slice of cols	A dataframe
A list/slice of rows	One col	A series, where the values are from the column, the rownames are from the row selection. The name attribute of the series now contains the colname
One row	A list/slice of cols	A series, where the values are from the row, the rownames are from the column selection and the name attribute of the series now contains the rowname
One row	One col	A scalar, like an integer or a string

- NB! A list/slice containing only one name is still a list/slice

`df1.loc[:,:]` or `df1.iloc[:, :]`

	10	13	12	17
3	A	153	55	10
2	B	160	70	11
1	C	150	60	12
0	D	190	80	13

	10	13	12	17
3	A	153	55	10
2	B	160	70	11
1	C	150	60	12
0	D	190	80	13

- `<class 'pandas.core.frame.DataFrame'>`

`df1.loc[:]` or `df1.iloc[:]`

	10	13	12	17
3	A	153	55	10
2	B	160	70	11
1	C	150	60	12
0	D	190	80	13

	10	13	12	17
3	A	153	55	10
2	B	160	70	11
1	C	150	60	12
0	D	190	80	13

- `<class 'pandas.core.frame.DataFrame'>`

`df1.loc[:,10:12]` or `df1.iloc[:,:-1]`

	10	13	12	17
3	A	153	55	10
2	B	160	70	11
1	C	150	60	12
0	D	190	80	13

	10	13	12	17
3	A	153	55	10
2	B	160	70	11
1	C	150	60	12
0	D	190	80	13

- `<class 'pandas.core.frame.DataFrame'>`

`df1.loc[[3,0]]` or `df1.iloc[[0,3]]`

	10	13	12	17
3	A	153	55	10
2	B	160	70	11
1	C	150	60	12
0	D	190	80	13

	10	13	12	17
3	A	153	55	10
2	B	160	70	11
1	C	150	60	12
0	D	190	80	13

- `<class 'pandas.core.frame.DataFrame'>`

`df1.loc[[2]]` or `df1.iloc[[1]]`

	10	13	12	17
3	A	153	55	10
2	B	160	70	11
1	C	150	60	12
0	D	190	80	13

	10	13	12	17
3	A	153	55	10
2	B	160	70	11
1	C	150	60	12
0	D	190	80	13

- `<class 'pandas.core.frame.DataFrame'>`

`df1.loc[2]` or `df1.iloc[1]`

	10	13	12	17
3	A	153	55	10
2	B	160	70	11
1	C	150	60	12
0	D	190	80	13

10	B
13	160
12	70
17	11
Name: 2	

- `<class 'pandas.core.series.Series'>`

`df1.loc[[2],[13]]` or `df1.iloc[[1],[1]]`

	10	13	12	17
3	A	153	55	10
2	B	160	70	11
1	C	150	60	12
0	D	190	80	13

	10	13	12	17
3	A	153	55	10
2	B	160	70	11
1	C	150	60	12
0	D	190	80	13

- `<class 'pandas.core.frame.DataFrame'>`

`df1.loc[[2],13]` or `df1.iloc[[-3],-3]`

	10	13	12	17
3	A	153	55	10
2	B	160	70	11
1	C	150	60	12
0	D	190	80	13

2	160
Name: 13	

- `<class 'pandas.core.series.Series'>`

`df1.loc[2,13]` or `df1.iloc[-3,1]`

	10	13	12	17
3	A	153	55	10
2	B	160	70	11
1	C	150	60	12
0	D	190	80	13

160

- `<class 'numpy.int64'>`

Name attribute

	10	13	12	17
3	A	153	55	10
2	B	160	70	11
1	C	150	60	12
0	D	190	80	13

- ```
  s1 = df1.loc[2]
  print(s1.name)   # On screen: 2
  ```
- ```
  s1 = df1.iloc[1]
  print(s1.name)   # On screen: 2
  ```
- ```
  s1 = df1.loc[:,13]
  print(s1.name)      # On screen: 13
  ```
- ```
  s1 = df1.iloc[:,1]
  print(s1.name)      # On screen: 13
  ```

Question, what is the result of

	0	2
3	A	153
2	B	160

- `print(df2.iloc[2, 2])`

Question, what is the result of

	0	2
3	A	153
2	B	160

- `print(df1.iloc[2, 2])`
- `IndexError: index 2 is out of bounds for axis 0 with size 2`

Update a selection

- You can use the selection mechanism also to change parts of a DataFrame

	10	13	12	17
3	A	153	55	10
2	B	160	70	11
1	C	150	60	12
0	D	190	80	13

- `df1.loc[[3,0]] = [['E', 100, 40, 20], ['F', 101, 41, 21]]`

	10	13	12	17
3	E	100	40	20
2	B	160	70	11
1	C	150	60	12
0	F	101	41	21

Selecting from a dataframe: boolean indexing

- Previously, we said that the selection of the columns and the rows in `df1.loc[`*`row_selection, col_selection`*`]` can be done via a slice, a list with names or one name.
- There is a fourth way: instead of giving a list of names, you can give a list or series with Boolean values (`True`, `False`)
  - This list with Boolean values has to have the same length as the number of rows
  - Then every row that corresponds with a `True` in the list is selected
- Instead of a list with Booleans, you can also use a series with Boolean values

Boolean indexing

- We want to select the green parts

	10	13	12	17
3	A	153	55	10
2	B	160	70	11
1	C	150	60	12
0	D	190	80	13

- With loc: `df1.loc[[3,2],[13,17]]`
- With iloc: `df1.iloc[:2,[1,-1]]`
- With Boolean indexing: `df1.loc[[True, True, False, False],[False, True, False, True]]`
- You could also mix and match:
  - `df1.loc[[True, True, False, False],[13,17]]`
  - `df1.loc[[3,2],[False, True, False, True]]`

Boolean indexing

- We want to select all rows, where the values in column 13 < 155

	10	13	12	17
3	A	153	55	10
2	B	160	70	11
1	C	150	60	12
0	D	190	80	13

- With loc: `df1.loc[[3,1]]`
- With iloc: `df1.iloc[::2]`
- With Boolean indexing: `df1.loc[df1.13 < 155]`
- You could also mix and match:
  - `df1.loc[[True, True, False, False],[13,17]]`
  - `df1.loc[[3,2],[False, True, False, True]]`
- You can also apply Boolean indexing to series

Selecting from a dataframe: shortcuts

- `df1.Name`
  - `df1.loc[:,'Name']`
- `df1['Average Weight']`
  - `df1.loc[:,'Average Weight']`
- `df1[['Name']]`
  - `df1.loc[:,['Name']]`
- `df1[['Name', 'Weight']]`
  - `df1.loc[:,['Name', 'Weight']]`
- `df1.Name['First']`
  - `df1.loc['First','Name']`
- `df1.Name[['First', 'Second']]`
  - `df1.loc[['First', 'Second'],'Name']`

Question: What will be printed by

	10	13	12	17
3	A	153	55	10
2	B	160	70	11
1	C	150	60	12
0	D	190	80	13

- `print(df1[[10,13]][[1,2]])`

Question: What will be printed by: `print(df1[[10,13]][[1,2]])`?

	10	13	12	17
3	A	153	55	10
2	B	160	70	11
1	C	150	60	12
0	D	190	80	13

- `df1 = df1[[10,13]]`

	10	13
3	A	153
2	B	160
1	C	150
0	D	190

- `print(df1[[1,2]])`
- An error

Selecting from a Series: shortcuts

- `s1[1]`
  - `df1.iloc[1]`
- `s1[:2]`
  - `s1.iloc[:2]`

Vectorized operations

- *Vectorized operations*: you apply the same operation to all elements of one or several columns, to a row or to a series
  - This is a very powerful part of Pandas
- The topics we will discuss are:
  - Broadcasting
  - Basic statistical operations are available as Series methods
  - Mapping a function to Series elements
  - Vectorized string methods
  - Applying a function to each row or to each column

Basic operations: broadcasting

- *Broadcasting* can be done for the whole dataframe or for one column or row
  - This works with scalars (integers, floats)
- Consider the following dataframe, called df:

	A	B
0	2	3
1	5	4
2	6	7

- The following give the same result:
  - `df += 1`
  - `df.A += 1; df.B +=1`

	A	B
0	3	4
1	6	5
2	7	8

Basic operations: broadcasting

- Instead of using broadcasting, you can also work with a more complex structure, as long as it has the same shape
  - For example: `df *= df:`

	A	B
0	4	9
1	25	16
2	36	49

df:

	A	B
0	2	3
1	5	4
2	6	7

  - The following also works: `df.A /= [1, 2, 3]`

	A	B
0	2.0	3
1	2.5	4
2	2.0	7

  - But the following gives an error: `df.A //= [1,2]`

```
# ValueError: operands could not be broadcast together with shapes (3,) (2,)
```

Basic statistical operations as Series methods

	A	B
0	2	3
1	5	4
2	6	7

- `mean, max, min, median, mode, sum, value_counts, std, ...` all work as methods of series
  - For example, `df.A.mean()` returns `4.333333333333333`
- These methods can be used in a vectorized way too
  - For example, `df.mean()` returns the series:

```
A 4.333333
B 4.666667
```

Mapping a function to all elements

df:

	A	B
0	2	3
1	5	4
2	6	7

- For example:
  - `df.A = df.A.map(lambda x: 3 * x)`
  - ```
    def triple(x):
        return x*3
    df.A = df.A.map(triple)
    ```

	A	B
0	6	3
1	15	4
2	18	7

- You can only apply this map method to a series
- If you want to apply a function to all elements (all cells) in the dataframe, you must use `applymap`:
  - For example: `df = df.applymap(lambda x: 3 * x)`

	A	B
0	6	9
1	15	12
2	18	21
    ```

Vectorized string methods

- Consider a Series called `s` that contains only strings
- You may want to apply the same string method, indexing or slicing to every element of this series
- We already saw that we can do this with the help of `s.map`
- But, remember that last week we learnt that an object is always build based on a class
- The basis of a string object is the class `str`
- By using this class name `str` we can directly call the methods in the class
- With this trick we can apply all methods that we know work for strings, slicing and indexing to the series
  - And because this is pandas, this will be broadcast to all elements of the series

Vectorized string methods

- The following pairs of code work exactly the same way:
  - `s.map(lambda x: x[::-1])`
    `s.str[::-1]`
  - `s.map(lambda x: x.lower())`
    `s.str.lower()`
  - `s.map(lambda x: x.count('a'))`
    `s.str.count('a')`
  - `s.map(lambda x: x.isupper())`
    `s.str.isupper()`
- And just like any other method, you can chain them with dot operators in between, if the result of the operation is a new series with only strings

Question

- What will the following program print:

```
import pandas as pd
s = pd.Series(['abc', 'xAef', 'aAgh', 'xyz'])
print(s.str.upper().str.count('A').mean())
```

a. 0.0

b. 0.5

c. 1.0

d. 1.5

Answer

- What will the following program print:

```
import pandas as pd
s = pd.Series(['abc', 'xAef', 'aAgh', 'xyz'])
print(s.str.upper().str.count('A').mean())
```

a. 0.0

b. 0.5

c. 1.0

d. 1.5

Answer: option c is correct.

The vectorized `in` operator

- The `in` operator is used to test whether a string is a substring of another string or a string is an element of a list (or tuple etc.)
- The vectorized version is called `isin`
- `isin` and `in` are not defined as methods in the str class
- The following work the same:
  - `s.map(lambda x: x in 'abcd')`
    `s.isin(list('abcd'))`
  - `s.map(lambda x: x not in 'abcd')`
    `~s.isin(list('abcd'))`
- The `~` operator works like the `not` operator but is vectorized
  - If you have a series with only Boolean values, and you apply the `~` operator all Trues will become False and all Falses will become True

Applying a function to each row/column

- Say you want a new row with the averages of each column:

```
df.loc['Av'] = df.apply(lambda x: x.mean(), axis = 0)
```

	A	B
0	2.000000	3.000000
1	5.000000	4.000000
2	6.000000	7.000000
Av	4.333333	4.666667

df:

	A	B
0	2	3
1	5	4
2	6	7

- Say you want a new column with the averages of each row:

```
df.loc[:,'Av'] = df.apply(lambda x: x.mean(), axis = 1)
```

	A	B	Av
0	2	3	2.5
1	5	4	4.5
2	6	7	6.5

Adding data to a dataframe

- In the previous slide we saw an example of adding a column to a dataframe
- When adding data to a column, the following applies:
  - A scalar value is broadcast to all rows
  - Collections of values (arrays, lists) must have a length and structure equal to the number of rows
  - A series of values added to a column will be fit into the dataframe according to the rownames of the dataframe and the indexnames of the series
  - Missing values may result if the two indices are not fully overlapping

Adding data to a dataframe

- When adding data to a row, the following applies:
  - A scalar value is broadcasted to all columns
  - Collections of values (arrays, lists) must have a length equal to the number of columns
  - A Series of values added to a row will be fit into the DataFrame according to the columnames of the dataframe and the indexnames in the series
  - Missing values may result if the two indices are not fully overlapping

Missing data

- A special value from Numpy is used to denote missing data: `np.nan`
  - Looks like `'NaN'` when printed, but it is not a string: it's something like `True`, `False`, and `None`
- Indicate missing values: `df.isna(),` creates the same shape as `df,` but with Trues and Falses
- Replace missing data with a value: `df.fillna(`value`)`
- Drop rows / columns with missing data: `df.dropna()`
- Missing data is sometimes automatically generated
  - e.g. by merging datasets with (partly) incompatible names/indices

Concatenating DataFrames

- Dataframes can be concatenated either vertically or horizontally:
  - Vertically: `pd.concat([df1, df2])`
  - Horizontally: `pd.concat([df1, df2], axis=1)`
- The resulting DataFrame has the union of the index / column labels in the direction of concatenation
- Missing values are created when the 'unionized' labels are not fully overlapping
- `axis=0`, is the default

Joining / merging DataFrames

- Data we want to use is often dispersed across several tables
- Example:
  - We have millions of tax records, including municipality of residence, in one table
  - We have each municipality and the province it is part of in another table
  - We want to calculate taxes paid per province
  - We'd have to match the province from the second table to each tax record in the first table, using the municipality name as a key for the matching, before we can calculate province-level statistics
  - We'd have to join / merge the DataFrames

Merging DataFrames

dftax

	Name	Income	Tax	City
0	A	100	10	Amsterdam
1	B	200	60	Rotterdam
2	C	150	30	London

dfprov

	City	Province
0	Amsterdam	Noord Holland
1	Rotterdam	Zuid Holland
2	Utrecht	Utrecht

Merging DataFrames: `on` parameter

- `print(dftax.merge(dfprov, how='left'))`
- `print(pd.merge(dftax, dfprov, how='left'))`
- `print(dftax.merge(dfprov, on= 'City', how='left'))`
- `print(dftax.merge(dfprov, left_on='City', right_on = 'City', how='left'))`

	Name	Income	Tax	City	Province
0	A	100	10	Amsterdam	Noord Holland
1	B	200	60	Rotterdam	Zuid Holland
2	C	150	30	London	NaN

- If both dataframes share some colnames, Pandas will use those to connect the data
  - If you want only some of the columns with the same name as the connection, you should use the `on` parameter
  - If the columns you want to use for the connection have different names, you have to use `right_on` and `left_on`
    - For example, if the city column in `dfprov` was called Cities you would need:

    `right_on ='Cities', left_on = 'City'`
  - The first dataframe is called left, the second one right

Merging DataFrames: `how` parameter

- `print(dftax.merge(dfprov, how='left'))`

	Name	Income	Tax	City	Province
0	A	100	10	Amsterdam	Noord Holland
1	B	200	60	Rotterdam	Zuid Holland
2	C	150	30	London	NaN

- `print(dftax.merge(dfprov, how='right'))`

	Name	Income	Tax	City	Province
0	A	100	10	Amsterdam	Noord Holland
1	B	200	60	Rotterdam	Zuid Holland
2	NaN	NaN	NaN	Utrecht	Utrecht

Grouping DataFrames (Split-Apply-Combine)

1. Splitting the data into groups based on some criteria
2. Applying a function to each group independently
3. Combining the results into a data structure

- Example:
  - In the tax records example earlier, we might split the data by province names, calculate total taxes paid for each province, then combine the resulting numbers into a Series indexed by province names

Grouping DataFrames

- ```
  dfcombined = dftax.merge(dfprov,  how='left')
  dfcombined.loc[:,'Province'] = dfcombined.loc[:,'Province'].fillna('Rest')
  # We replace the missing values with a string
  # As the groupby method ignores NaN values
  result = dfcombined.groupby('Province')['Tax'].sum()
  result = result.sort_values(ascending = False)
  # We want the highest taxed Province first
  ```

- The result will be:

Zuid Holland	60
Rest	35
Noord Holland	10

- You get the total tax with:
  ```
  print(result.sum())                # Onscreen 105
  ```
- And the tax for Noord Holland with:
  ```
  print(result['Noord Holland'])  # Onscreen: 10
  ```
  ```